COURSE NAME: DEEP LEARNING

COURSE CODE: CSA4708

NAME: THAMIZHARASAN S

REG NO: 192424087

Gradient Descent, SGD, Momentum and Adam Comparison

Question 1: Student Performance Prediction

A college develops a neural network to predict student examination scores using attendance and internal assessment marks. The model trained using Gradient Descent shows a slow decrease in loss, while the model trained using SGD shows fluctuations in loss across epochs.

Analyze the training behavior of both optimizers by implementing the models using TensorFlow and Keras, storing loss values using NumPy, and plotting the loss curves using Matplotlib. Identify the possible reasons for the differences in convergence.

Packages: NumPy, TensorFlow, Keras, Matplotlib

AIM:

To compare Batch Gradient Descent and SGD for student score prediction using TensorFlow, NumPy, and Matplotlib.

PROCEDURE:

1. Import the required libraries.
2. Create student attendance and internal marks data.
3. Build a neural network using Keras.
4. Train models using Batch Gradient Descent and SGD.
5. Store loss values using NumPy.

PROGRAM CODE:

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
X = np.array([[60, 50], [70, 60], [80, 70],
              [90, 80], [95, 90]], dtype=float) / 100
y = np.array([50, 60, 70, 80, 90], dtype=float)
tf.keras.utils.set_random_seed(42)
def model():
    return keras.Sequential([
        layers.Input(shape=(2,)),
        layers.Dense(8, activation='relu'),
        layers.Dense(1)
    ])
m1 = model()
m2 = model()
m2.set_weights(m1.get_weights())
m1.compile(optimizer=keras.optimizers.SGD(0.01),
           loss='mse')
m2.compile(optimizer=keras.optimizers.SGD(0.05),
           loss='mse')
```

```
h1 = m1.fit(X, y, epochs=50, batch_size=5, verbose=0)

h2 = m2.fit(X, y, epochs=50, batch_size=1, verbose=0)

loss1 = np.array(h1.history['loss'])

loss2 = np.array(h2.history['loss'])

plt.plot(loss1, label='Batch GD')

plt.plot(loss2, label='SGD')

plt.xlabel('Epochs')

plt.ylabel('Loss (MSE)')

plt.legend()

plt.grid()

plt.show()

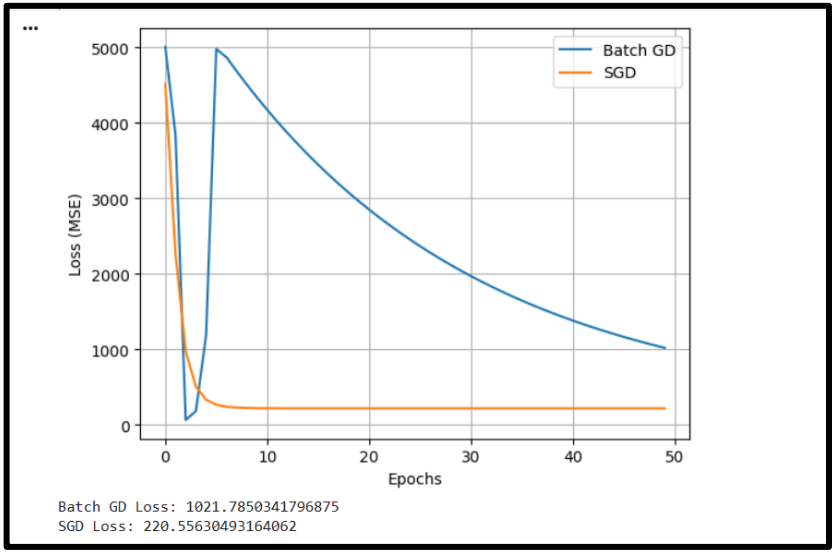
print("Batch GD Loss:", loss1[-1])

print("SGD Loss:", loss2[-1])
```

INPUT:

- Attendance: 60, 70, 80, 90, 95
- Internal marks: 50, 60, 70, 80, 90
- Epochs: 50
- Loss function: Mean Squared Error (MSE)

OUTPUT:



RESULT:

Batch Gradient Descent generally produces smoother loss reduction, while SGD shows more fluctuations due to individual-sample updates. The experiment demonstrates how batch size and learning rate affect convergence.

Question 2: House Price Prediction

A real-estate company trains a neural network to predict house prices. During training, the loss decreases rapidly at first but fluctuates significantly when SGD is used with a high learning rate. When a smaller learning rate is used, training becomes more stable but slower.

Analyze how the learning rate affects SGD convergence by training the model with different learning rates using TensorFlow and Keras. Use NumPy to organize the loss values and Matplotlib to compare the loss curves. Determine which learning-rate setting provides a better balance between convergence speed and stability.

Packages: NumPy, TensorFlow, Keras, Matplotlib

AIM:

To analyze the effect of different learning rates on SGD convergence for house price prediction using TensorFlow, Keras, NumPy, and Matplotlib.

PROCEDURE:

1. Import the required libraries.
2. Create sample house area and price data.
3. Build a neural network using Keras.
4. Train models using SGD with different learning rates.
5. Store loss values using NumPy.
6. Plot and compare the loss curves.

PROGRAM CODE:

```
import numpy as np

import tensorflow as tf

import matplotlib.pyplot as plt

from tensorflow import keras
```

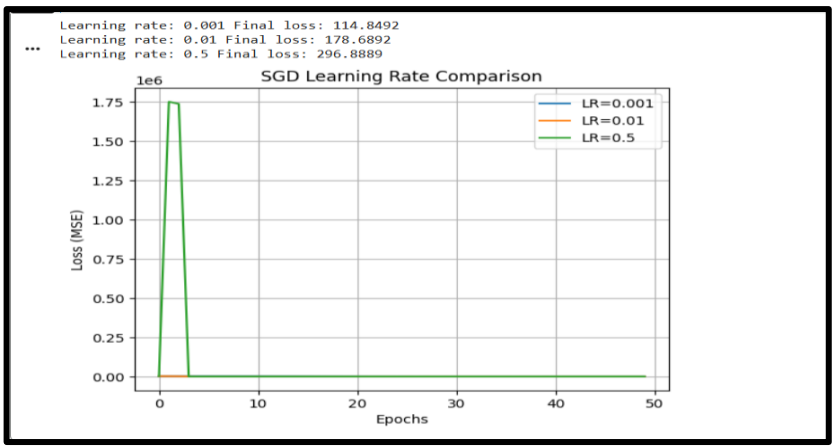
```
from tensorflow.keras import layers
X = np.array([[500], [800], [1000], [1200],
              [1500], [1800]], dtype=float) / 2000
y = np.array([20, 32, 40, 48, 60, 72], dtype=float)
rates = [0.001, 0.01, 0.5]
losses = {
tf.keras.utils.set_random_seed(42)
for lr in rates:
    model = keras.Sequential([
        layers.Input(shape=(1,)),
        layers.Dense(8, activation='relu'),
        layers.Dense(1)
    ])
    model.compile(
        optimizer=keras.optimizers.SGD(learning_rate=lr),
        loss='mse'
    )
    history = model.fit(X, y, epochs=50, verbose=0)
    losses[lr] = np.array(history.history['loss']
    print("Learning rate:", lr,
          "Final loss:", round(losses[lr][-1], 4))
for lr in rates:
    plt.plot(losses[lr], label=f'LR={lr}')
```

```
plt.xlabel("Epochs")
plt.ylabel("Loss (MSE)")
plt.title("SGD Learning Rate Comparison")
plt.legend()
plt.grid()
plt.show()
```

INPUT:

- House areas: 500, 800, 1000, 1200, 1500, 1800 sq. ft.
- House prices: 20, 32, 40, 48, 60, 72 lakhs.
- Learning rates: 0.001, 0.01, 0.5.
- Epochs: 50.

OUTPUT:



RESULT:

The learning rate affects SGD convergence. A moderate learning rate provides a good balance between training speed and stability.

Question 3: Handwritten Digit Classification

An image-recognition system uses a neural network to classify handwritten digits. The model trained with SGD takes longer to reach a low training loss, whereas the model trained with Momentum converges more quickly.

Analyze the performance differences between SGD and Momentum by training both models under the same experimental conditions. Use NumPy to compare epoch-wise losses and Matplotlib to visualize convergence. Explain how Momentum influences updates when gradients consistently point in a similar direction.

Packages: NumPy, TensorFlow, Keras, Matplotlib

AIM:

To compare SGD and Momentum for handwritten digit classification using TensorFlow, Keras, NumPy, and Matplotlib.

PROCEDURE:

1. Import the required libraries.
2. Load the MNIST handwritten digit dataset.
3. Build identical neural networks.
4. Train using SGD and Momentum under the same conditions.
5. Store epoch-wise losses using NumPy.
6. Plot and compare the loss curves.

PROGRAM CODE:

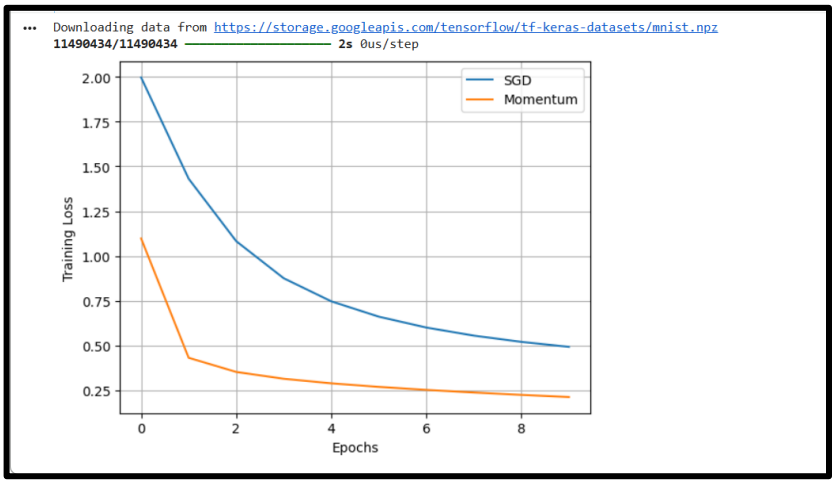
```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
tf.keras.utils.set_random_seed(42)
(X, y), _ = keras.datasets.mnist.load_data()
X = X[:10000].astype("float32") / 255.0
y = y[:10000]
def create_model():
    return keras.Sequential([
        layers.Input(shape=(28, 28)),
        layers.Flatten(),
        layers.Dense(128, activation="relu"),
        layers.Dense(10, activation="softmax")
    ])
m1 = create_model()
m2 = create_model()
m2.set_weights(m1.get_weights())
m1.compile(optimizer=keras.optimizers.SGD(0.01),
           loss="sparse_categorical_crossentropy")
m2.compile(optimizer=keras.optimizers.SGD(
```

```
    0.01, momentum=0.9),
    loss="sparse_categorical_crossentropy")
h1 = m1.fit(X, y, epochs=10, batch_size=128, verbose=0)
h2 = m2.fit(X, y, epochs=10, batch_size=128, verbose=0)
loss_sgd = np.array(h1.history["loss"])
loss_momentum = np.array(h2.history["loss"])
plt.plot(loss_sgd, label="SGD")
plt.plot(loss_momentum, label="Momentum")
plt.xlabel("Epochs")
plt.ylabel("Training Loss")
plt.legend()
plt.grid()
plt.show()
```

INPUT:

- Dataset: MNIST handwritten digits.
- Images: 28 × 28 pixels.
- Optimizers: SGD and Momentum (0.9).
- Epochs: 10.
- Batch size: 128.

OUTPUT:



RESULT:

Momentum generally accelerates convergence by accumulating past gradients, reducing oscillations when gradients consistently point in a similar direction.

Question 4: Disease Risk Classification

A healthcare analytics team builds a neural network to classify patient records into risk categories. The team observes that Adam reduces training loss quickly, while ordinary Gradient Descent requires more epochs to achieve a similar loss value.

Analyze the convergence behavior of Gradient Descent and Adam by training comparable Keras models with TensorFlow. Record the loss values using NumPy and plot the results using Matplotlib. Evaluate the differences in convergence speed and training stability, and explain why Adam's adaptive learning rates may be beneficial.

Packages: NumPy, TensorFlow, Keras, Matplotlib

AIM:

To compare Gradient Descent and Adam for disease risk classification using TensorFlow, Keras, NumPy, and Matplotlib.

PROCEDURE:

- Import the required libraries.
- Create sample patient data and risk labels.
- Build identical neural network models.
- Train using Gradient Descent and Adam.
- Store loss values using NumPy.
- Plot and compare convergence curves.

PROGRAM CODE:

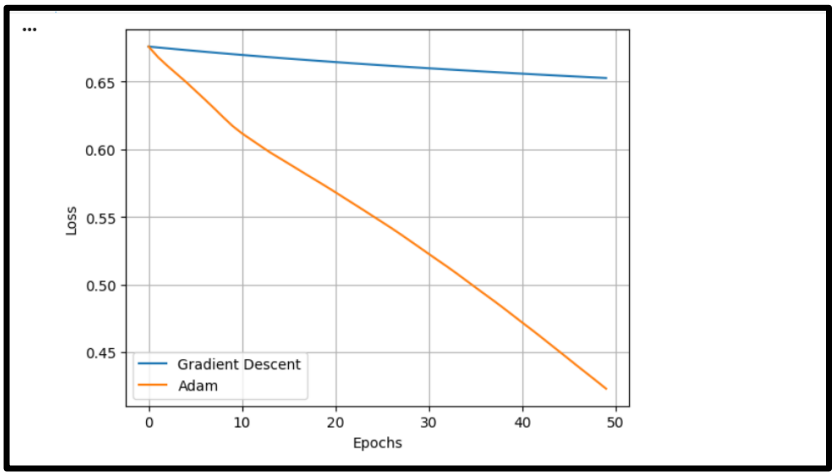
```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
tf.keras.utils.set_random_seed(42)
X = np.array([[25, 110], [35, 120], [45, 140],
              [55, 150], [65, 160], [75, 180]],
             dtype=float)
y = np.array([0, 0, 0, 1, 1, 1])
X = X / np.max(X, axis=0)
def create_model():
    return keras.Sequential([
        layers.Input(shape=(2,)),
        layers.Dense(8, activation="relu"),
        layers.Dense(1, activation="sigmoid")
    ]
m1 = create_model()
m2 = create_model()
m2.set_weights(m1.get_weights())
m1.compile(optimizer=keras.optimizers.SGD(0.01),
           loss="binary_crossentropy")
```

```
m2.compile(optimizer=keras.optimizers.Adam(0.01),
        loss="binary_crossentropy"
h1 = m1.fit(X, y, epochs=50, verbose=0)
h2 = m2.fit(X, y, epochs=50, verbose=0)
gd_loss = np.array(h1.history["loss"])
adam_loss = np.array(h2.history["loss"])
plt.plot(gd_loss, label="Gradient Descent")
plt.plot(adam_loss, label="Adam")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid()
plt.show()
```

INPUT:

- Features: Patient age and blood pressure.
- Target: Risk category (0 = low, 1 = high).
- Epochs: 50.
- Optimizers: SGD and Adam.

OUTPUT:



RESULT:

Adam generally converges faster because it adapts learning rates using past gradients. Gradient Descent may require more epochs, while Adam often provides faster and stable training.

Question 5: Optimizer Comparison for Sales Forecasting

A retail company develops a neural network to forecast weekly sales. Four versions of the model are trained using Gradient Descent, SGD, Momentum and Adam. The models show different training-loss patterns, convergence speeds and validation performance.

Analyze and compare the four optimizers by training the models using TensorFlow and Keras under comparable conditions. Use NumPy to store and compare training and validation losses, and Matplotlib to plot the loss curves. Identify which optimizer provides the most suitable combination of convergence speed, stability and validation performance, and justify the selection using experimental evidence.

Packages: NumPy, TensorFlow, Keras, Matplotlib

AIM:

To compare Gradient Descent, SGD, Momentum, and Adam for weekly sales forecasting using TensorFlow, Keras, NumPy, and Matplotlib.

PROCEDURE:

1. Import the required libraries.
2. Generate sample weekly sales data.
3. Build identical neural network models.
4. Train models using four optimizers.
5. Store training and validation losses using NumPy.
6. Plot loss curves and compare performance.

PROGRAM CODE:

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
from tensorflow.keras import layers
tf.keras.utils.set_random_seed(42)
X = np.arange(1, 101, dtype=float).reshape(-1, 1)
y = (100 + 5 * X.flatten() +
     10 * np.sin(X.flatten() / 5))
X = X / 100
y = y / 600
X_train, X_val = X[:80], X[80:]
y_train, y_val = y[:80], y[80:]
optimizers = {
    "Gradient Descent": keras.optimizers.SGD(0.01),
    "SGD": keras.optimizers.SGD(0.01),
    "Momentum": keras.optimizers.SGD(0.01, momentum=0.9),
    "Adam": keras.optimizers.Adam(0.01)
}
train_losses = {}
val_losses = {}
```

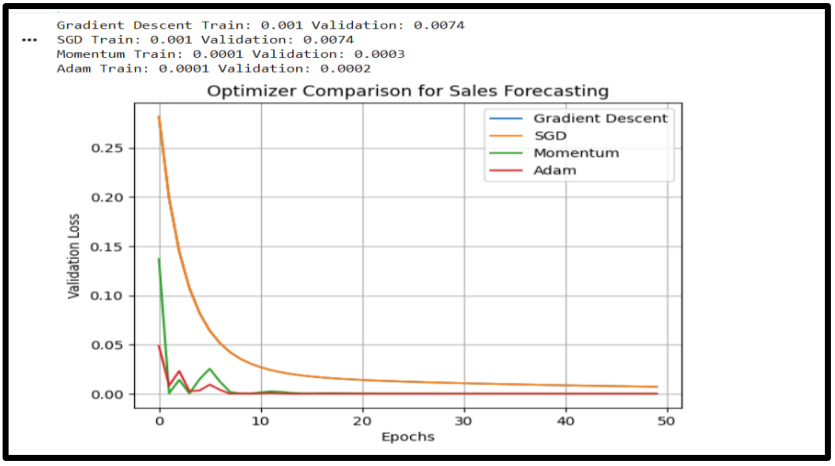
```
for name, opt in optimizers.items():
    tf.keras.backend.clear_session()
    tf.keras.utils.set_random_seed(42)
    model = keras.Sequential([
        layers.Input(shape=(1,)),
        layers.Dense(16, activation="relu"),
        layers.Dense(1)
    ])
    model.compile(optimizer=opt, loss="mse")
    history = model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=50, batch_size=16, verbose=0
    )
    train_losses[name] = np.array(history.history["loss"])
    val_losses[name] = np.array(history.history["val_loss"])
    print(name,
          "Train:", round(train_losses[name][-1], 4),
          "Validation:", round(val_losses[name][-1], 4))
for name in optimizers:
    plt.plot(val_losses[name], label=name)
plt.xlabel("Epochs")
plt.ylabel("Validation Loss")
```

```
plt.title("Optimizer Comparison for Sales Forecasting")
plt.legend()
plt.grid()
plt.show()
```

INPUT:

- Input: Weekly sales observations.
- Target: Predicted sales.
- Optimizers: Gradient Descent, SGD, Momentum, Adam.
- Epochs: 50.
- Loss function: Mean Squared Error (MSE).

OUTPUT:



RESULT:

Adam generally converges faster, while the best optimizer is determined by the lowest validation loss and stable training.